Unidad 1 / Escenario 2

Lectura fundamental

Estructuras de control de flujo de programa

Contenido

1 Buenas prácticas de programación 1

2 Estructuras de decisión 1

3 Estructuras de repetitivos 9

4 Sentencias de interrupción de ciclos 18

5 Usos especiales de variables en ciclos 20

6 División de cadenas 23

7 Problemas propuestos 25

8 Extras 30

Palabras clave: programación, lenguajes de programación, imperativo, orientado a objetos, Java, ciclos, *for*, *while*, *if*, *else* .

1. Buenas prácticas de programación

Tal vez uno de los aspectos que habla más de un programador al ver sus creaciones es qué tanto respeta las buenas prácticas de programación que son conocidas y aprobadas por la comunidad de programadores. Aunque las prácticas de codificación dependen un poco del lenguaje de programación usado y de las convenciones que se tengan en cada equipo de trabajo, hay algunas que son universales; un ejemplo de ello es el correcto espaciado en los códigos fuentes. Aunque hay numerosas variaciones de indentado consideradas como buenas prácticas por diferentes equipos y comunidades, en este documento se intenta respetar pocas variaciones, cuyos ejemplos se pueden ver en la sección 8.

Los estándares sobre la forma de documentar correctamente, por lo general, están dados por las compañías o personas que crean las herramientas o que coordinan los equipos de desarrollo. En el caso de Java, Oracle tiene bien detallado cómo hacerlo (Oracle, 1999b). Documentar bien es de suma importancia por varias razones; entre ellas, que una buena documentación del código, respetando estándares, puede, con ayuda de algunas herramientas, generar documentación de manera pseudoautomática, como en el caso de Java.

Otro objetivo de documentar correctamente es facilitar el proceso de lectura de código, lo cual ayuda al propio autor, su equipo de trabajo y a futuros colaboradores a entenderlo y, llegado el caso, a modificarlo o extenderlo fácil y correctamente. Por otro lado, las buenas prácticas de programación, como el correcto espaciado (indentado) y el nombramiento de variables/clases/métodos depende tanto del equipo de desarrollo como de la empresa o entidad creadora del lenguaje de programación, o de comunidades o empresas influyentes creadas o que funcionen alrededor de él. En el caso de Java, son grandes referentes las recomendaciones de dos reconocidas multinacionales: Oracle (1999a) y Google (2018). Una correcta codificación y nombramiento de variables/clases/métodos en inglés, junto con uno de varios ejemplos de buena indentación y buena documentación, son suficientes. Para más detalles, se sugiere ver las fuentes mencionadas.

2. Estructuras de decisión

En cuanto a estructuras de control de programa, las más comunes en los lenguajes de programación son las que evalúan una condición y una expresión, y, dependiendo de su valor de verdad, el flujo de programa se va por un camino u otro. En las estructuras de control que tienen algunos lenguajes de propósito general, incluido Java, es válido omitir el uso de llaves { y } en el caso de que haya una sola instrucción dentro del bloque en cuestión.

2.1. *if*

El uso del inglés es importante en el nombramiento de variables, en los mensajes a la salida estándar y, en general, en todo el proceso de codificación. La razón es que vivimos y trabajamos en un ambiente globalizado y cuando necesitemos apoyo o tengamos la posibilidad de trabajar con o para equipos internacionales es muy bueno tener como costumbre la programación usando el inglés para el nombramiento de elementos en el código y también para la documentación. A continuación, el tema se introduce con un ejemplo de la estructura de control de flujo de programa *if*.

2.1.1. Ejemplo 1

En el ejemplo de Código 1 se aprecia el uso del condicional hecho primero con llaves y luego omitiéndolas, ya que hay una sola instrucción dentro del *if*. Se aclara que cuando hay más de una instrucción dentro del bloque del *if* no se pueden omitir las llaves, ya que a pesar de que el programa compilaraá correctamente, no hará lo deseado, que es ejecutar las instrucciones que creemos están dentro del bloque.

```
public class IFExample
{
    public static void main(String[] args)
    {
        int n = 3;
        if( n > 0 && n % 2 == 1)
        {
            System.out.println(n+" is positive and odd .");
        }
        System.out.println("This message is always printed.");

        //The same code without curly brackets:
        if( n > 0 && n % 2 == 1)
            System.out.println(n+" is positive and odd .");
         System.out.println("This message is always printed.");
    }
}
```

Código 1: ejemplo de uso del *if*.

2.2. *if-else*

Aunque es válido usar la estructura de control *if* sola con una serie de acciones a realizar si una condición es verdadera, esta estructura también puede ser usada cuando la condición es falsa. Recuerde que dentro de la estructura *if* la expresión da únicamente dos posibles resultados: un valor falso o uno verdadero. Por lo tanto, cuando la expresión es evaluada y da verdadero, se ejecuta lo que está dentro de las llaves del *if*, y si da falso, se ejecuta lo que está dentro de las llaves del bloque *else*.

2.2.1. Ejemplo 2

En el Código 2, el cual es similar al Código 1, se puede ver un ejemplo de uso de la estructura de control *if* – *else*.;en este se puede apreciar su uso con y sin llaves, tanto en la parte *if* como en la parte del *else*. Lo que nunca se puede pasar por alto es que solo se pueden omitir las llaves cuando hay una sola instrucción en ese bloque de código.

```
import java.util.Scanner; public
class IfElseExample
{
    public static void main(String[] args){

        Scanner sn = new Scanner( System.in );
        System.out.print("Type an integer: "); int n
        = sn.nextInt();
        if( n > 0 && n % 2 == 1)
        {
            System.out.println(n+" is positive and odd .");
        }
        else
        {
            System.out.println(n+" is not positive or is even");
        }
        System.out.println("This message is always printed.");

        //The same code without curly brackets:
        if( n > 0 && n % 2 == 1)
            System.out.println(n+" is positive and odd .");
        else
              System.out.println(n+" is not positive or is even");
        System.out.println("This message is always printed.");

        //The same code without curly brackets only in the if:
        if( n > 0 && n % 2 == 1)
        {
            System.out.println(n+" is positive and odd .");
        }
        else
            System.out.println(n+" is not positive or is even");
        System.out.println("This message is always printed.");

        //The same code without curly brackets only in the else:
        if( n > 0 && n % 2 == 1)
            System.out.println(n+" is positive and odd .");
        else
        {
            System.out.println(n+" is not positive or is even");
        }
        System.out.println("This message is always printed.");

        sn.close();
    }
}
```

Código 2: ejemplo de uso del *if-else*.

2.3. *if-else if-else*

En algunas situaciones, el programador se puede enfrentar a la tarea de hacer una de varias cosas en función de más de una condición. Cuando esto pasa, la estructura de control *if-else* se puede usar varias veces en una misma construcción. En este caso, se puede omitir la última condición si no se necesita.

2.3.1. Ejemplo 3

En el Código 3 hay un ejemplo del uso del condicional *if* varias veces con diversas condiciones. En este código se podría haber puesto una condición adicional en el último *else* para verificar que el número leído es cero. El Código 3 muestra un programa que toma como entrada un número entero de la entrada estándar y despliega uno de los siguientes mensajes (en inglés), según sea el caso:

- El número es positivo e impar.
- El número es positivo y par.
- El número es negativo e impar.
- El número es negativo y par.
- n es 0.

Adicional a lo anterior, el programa siempre despliega el mensaje: "Este mensaje se imprime siempre" (en inglés: "*This message is always printed*").

```
import java.util.Scanner; public
class IfElseIfExample{
    public static void main(String[] args){
        Scanner sn = new Scanner( System.in );
        System.out.print("Type an integer: ");
        int n = sn.nextInt();
        if( n > 0 && n % 2 == 1){
            System.out.println(n+" is positive and odd.");
        }
        else if( n > 0 && n % 2 == 0){
            System.out.println(n+" is positive and even.");
        }
        else if( n < 0 && n % 2 == 1){
            System.out.println(n+" is negative and odd.");
        }
        else if( n < 0 && n % 2 == 0){
            System.out.println(n+" is negative and even.");
        }
        else{
            System.out.println("n is "+n+".");
        }
        System.out.println("This message is always printed.");

        //The same code without curly brackets:
        if( n > 0 && n % 2 == 1)
            System.out.println(n+" is positive and odd."); else if( n >
        0 && n % 2 == 0)
            System.out.println(n+" is positive and even."); else if( n <
        0 && n % 2 == 1)
            System.out.println(n+" negative and odd.");
        else if( n < 0 && n % 2 == 0)
            System.out.println(n+" negative and even.");
        else
            System.out.println("n is "+n+"."); System.out.println("This
        message is always printed."); sn.close();
    }
}
```

Código 3: ejemplo de uso del *if-else-if-else*.

2.4. *Switch*

En algunas ocasiones, se necesita tomar una decisión acerca de una serie de posibles opciones de acuerdo con un valor dado: entero, carácter o cadena. En estas situaciones, se pueden usar sentencias *if* anidadas o consecutivas, o sentencias *if-else-if-else*. Otra forma es usar la estructura *switch*, la cual permite hacer una opción de un conjunto de posibilidades según el valor que tiene una variable tipo entero, carácter o cadena.

2.4.1. Ejemplo 4

En el Código 4 se puede ver el uso del *switch* y un código que produce el mismo resultado, pero usando *if-else*. Se recomienda su análisis detallado y su seguimiento paso a paso para un mayor entendimiento.

```
import java.util.Scanner;
public class SwitchExample1{
    public static void main( String args[]){
        Scanner sn = new Scanner( System.in );
        System.out.print("Type an integer number between 1 and 5: ");
        int option = sn.nextInt();
        switch ( option) {
            case 1:
                System.out.println("You typed 1");
                break;
            case 2:
                System.out.println("You typed 2");
                break;
            case 3:
                System.out.println("You typed 3");
                break;
            case 4:
                System.out.println("You typed 4");
                break;
            case 5:
                System.out.println("You typed 5");
                break;
            default:
                System.out.println("You did not type a number beetween 1 and 5");
                break;
        }
        if( option == 1 )
            System.out.println("You typed 1");
        else if( option == 2 )
            System.out.println("You typed 2");
        else if( option == 3 )
            System.out.println("You typed 3");
        else if( option == 4 )
            System.out.println("You typed 4");
        else if( option == 5 )
            System.out.println("You typed 5");
        else
        System.out.println("You did not type a number beetween 1 and 5");
        sn.close();
    }
}
```

Código 4: ejemplo de uso del *switch*.

En un *switch*, después de que se encuentra una coincidencia en el valor de su variable, se empieza a ejecutar el código dentro del *switch* hasta que se encuentre la instrucción *break*. Después de hallar dicha instrucción, el flujo de programa sigue después del *switch*. Estas instrucciones break no son obligatorias en cada opción; se pueden omitir algunas. En el Código 5 se aprecia el uso del *switch* dejando de lado algunas instrucciones *break* y un código que produce el mismo resultado, pero usando *if-else*.

```
import java.util.Scanner;
public class BooleanExample{
    public static void main( String args[ ]){
        Scanner sn = new Scanner( System.in );
        System.out.print("Type an integer number between 1 and 5: ");
        int option = sn.nextInt();
        switch (option) {
            case 1:
                System.out.println("You typed 1");
            case 2:
                System.out.println("You typed 2 or 1");
            case 3:
                System.out.println("You typed 3 or 2 or 1");
            case 4:
                System.out.println("You typed 4 or 3 or 2 or 1");
            case 5:
                System.out.println("You typed 5 or 4 or 3 or 2 or 1");
                break;
            default:
                System.out.println("You did not type a number beetween 1 and 5");
                break;
        }
        if( option == 1 )
            System.out.println("You typed 1");
        if( option == 1 || option == 2 )
                System.out.println("You typed 2 or 1");
           if( option == 1 || option == 2 || option == 3 )
            System.out.println("You typed 3 or 2 or 1");
        if( option == 1 || option == 2 || option == 3 || option == 4 )
            System.out.println("You typed 4 or 3 or 2 or 1");
        if( option == 1 || option == 2 || option == 3 || option == 4 || option == 5 )
            System.out.println("You typed 5 or 4 or 3 or 2 or 1");
        if(!( option == 1 || option == 2 || option == 3 || option == 4 || option == 5 ))
            System.out.println("You did not type a number beetween 1 and 5");
        sn.close();
    }
}
```

Código 5: ejemplo de uso del *switch* omitiendo algunos *break*.

3. Estructuras de repetitivos

Una de las principales ventajas de saber programar es que se puede crear *software* que haga de una manera rápida y eficiente cálculos y operaciones que a las personas les tomaría mucho tiempo. En muchas de estas tareas es necesario realizar procesos similares una y otra y otra vez hasta que algo ocurra (un número predeterminado de veces o mientras algo siga pasando). Es por esto que prácticamente todos los lenguajes de programación tienen estructuras de control para hacer de una manera u otra repeticiones. Las más comunes son los ciclos *for*, *while* y *do–while*.

3.1. Ciclo for

Los ciclos *for* son una instrucción muy usada en programación. Es recomendada para cuando se quiere repetir un bloque de instrucciones en el código un número determinado de veces. Consta fundamentalmente de seis partes, cada una señalada entre comentarios /* y */ en el Código 6.

1. /*1*/: la primera parte la componen las instrucciones que están antes del ciclo y se ejecuta una sola vez. En el ejemplo del Código 6 es solo la instrucción: int max = 10;
2. /*2*/: la segunda parte está dentro de la estructura *for*(aquí está´;;) y se ejecuta una sola vez al inicio del ciclo *for*. En el ejemplo del Código 6 es solo la instrucción: int i = 0
3. /*3*/: la tercera parte es una operación que debe dar como resultado un valor de verdad (falso o verdadero) y es la que controla hasta qué momento se deben hacer las instrucciones del ciclo *for*(; aquí está ;). En el ejemplo del Código 6 es la expresión i <max
4. /*4*/: la cuarta parte es el cuerpo del ciclo *for*, es decir, lo que está dentro: las instrucciones que queremos repetir cierto número de veces. Estas instrucciones empiezan su ejecución cada vez que la expresión en la parte 3 es evaluada como verdadera. En el ejemplo del Código 6 es la instrucción de impresión System.out.println(i);
5. /*5*/: la quinta parte es la instrucción que se ejecuta después de cada repetición (ejecución) del bloque de código dentro del ciclo. Es decir, se ejecuta siempre después de la cuarta parte: *for* (;;aquí está). En el ejemplo del Código 6 es ++i
6. /*6*/: finalmente, la sexta parte es sencillamente lo que está después del ciclo. En el Código 6 esto es el mismo ciclo, pero sin los comentarios, resaltando las 6 partes.

El orden de instrucciones en el ciclo *for* viene dado por: la parte 1, que se ejecuta una única vez; luego la parte 2, que también se ejecuta una única vez; después la parte 3, que es una evaluación de una expresión booleana, y, de ser esta cierta, se ejecutan las partes 4 y 5. Después de ejecutar las partes 4 y 5, vuelve a evaluarse la expresión de la parte 3 y, si sigue siendo cierta, se vuelven a ejecutar las partes 4 y 5, y así sucesivamente hasta que la parte 3 de falso al ser evaluada, en cuyo caso la siguiente parte a ejecutarse es la parte 6, que es la instrucción después del ciclo *for*. La secuencia de ejecución de partes, si pudiéramos escribirla, sería algo como: 1,2,3,4,5, 3,4,5 3,4,5 3,4,5 3,4,5,...,3,6.

El ciclo *for* es entonces recomendado para ser usado en situaciones en donde se requiere hacer cierto número de iteraciones (otro nombre para las repeticiones) un determinado número de veces, el cual puede ser fijo o producto de alguna operación o lectura de datos. También se recomienda usar el ciclo *for* para iterar sobre secuencias de números enteros con separaciones homogéneas, en especial si es para iterar sobre arreglos o colecciones de elementos.

3.1.1. Ejemplo 5

```
public class ForExample{
    public static void main( String args[]){

        int max = 10;
        /*1*/
        for(  /*2*/ int i = 0;  /*3*/  i < max;  /*5*/  ++ i ){

            /*4*/
            System.out.println(i);
        }
        /*6*/

        for( int i = 0; i < max; ++ i )
        {
            System.out.println(i);
        }
    }
}
```

Código 6: ejemplo de uso del ciclo *for*.

En el ejemplo del Código 6 se muestra un *for* que itera desde 0 hasta *max* − 1. Es común que la lectura de estos ciclos a la hora de explicarlos o leerlos sea: para *i* variando desde 0 hasta menor que *max*, incrementado en 1, haga.... Poder leer código es también una habilidad que todo programador debe tener.

3.1.2. Ejemplo 6

En muchos algoritmos y situaciones conviene usar ciclos anidados para resolver problemas. En el Código 7 hay un ejemplo de ello. Este código muestra un programa para imprimir las primeras ***m*** tablas de multiplicar, cada una con ***n*** términos.

```
public class ForExample{
    public static void main( String args[]){
        int m = 10;
        int n = 5;

        for( int i = 0; i < m; ++i )
        {
            System.out.println("Multiplication table for: "+i);
            for( int j = 0; j < n; ++ j )
            {
                System.out.println(i+" x "+j+" = "+(i*j));
            }
        }
    }
}
```

Código 7: ejemplo de uso del ciclos *for* anidados.

3.1.3. Ejemplo 7: Usos exóticos de ciclos *for*

En la descripción del funcionamiento del ciclo *for* en la sección 3.1 se mostraron 6 partes en el ciclo *for*. Los usos más comunes de las partes 2, 3 y 5 son declaraciones e inicializaciones para la parte 2, una comparación para la parte 3 y un incremento para la parte 5. Aunque mantener esas tradiciones es una buena práctica de programación, la estructura del ciclo *for* da libertades al programador para idearse soluciones o aproximaciones a soluciones de problemas de manera creativa y corta, aunque muchas veces muy poco legibles, como se muestra en el Código 8. En este, en el primer *for* se usan tres contadores y en el segundo *for* se muestra una manera muy poco legible de pasar números de decimal a binario. Aunque en general una buena práctica de programación es hacer el código escrito lo más legible posible (algo que no está presente en el Código 8), un buen ejercicio para repasar fundamentos de operadores, operaciones y ciclo *for* es hacer un seguimiento paso a paso de este código y entender qué hace cada instrucción y cómo contribuye a que, al final del programa, en la variable *binary* quede almacenado un valor entero que se ve como la representación binaria de la variable ***x***.

```
public class ForExample{
    public static void main( String args[]){
        int n = 15;
        int a = 99;
        int b = 999;
        System.out.println("i\tj\tk");
        for( int i = a,j = b, k = 0; k < n; ++i, --j, ++k)
            System.out.println(i+"\t"+j+"\t"+k);
        long x = 123456, tmp, binary, p;
        for( tmp = x, binary = 0,
                p = 1; (tmp|0) != 0 ;binary += (tmp & 1)*p, p =
                (p << 4)-(p << 2)-(p << 1), tmp >>= 1 );
        System.out.println(x+" in binary is: "+binary);
    }
}
```

Código 8: ejemplo de usos exóticos de ciclos for.

3.1.4. Ejemplo 8: Ciclo *for* para iterar sobre un conjunto

Uno de los tantos usos de los ciclos es iterar sobre subconjuntos de un posible conjunto representado utilizando un solo *integer*, como se expuso en el Escenario 1. En el Código 9 se muestra cómo iterar sobre todos los subconjuntos del conjunto {a, b, c, d, e, f}

```
import java.util.Arrays;
public class StringBitsAndSetsExample {
    public static void main(String[] args){
        String s = "abcdef";
        int n = s.length();
        int limit = 1<<n;
        String arr[] = s.split("");
        System.out.println("Print all subsets of: "+Arrays.toString(arr));
        for( int mask = 0; mask < limit; ++ mask )
        {
            System.out.print("{ ");
            for (int bit = 0; bit < n; bit++)
            {
                boolean isBitOn = (mask & (1<<bit) ) > 0;
                if( isBitOn )
                {
                    System.out.print(s.charAt(bit)+" "); }
            }
            System.out.println("}");

        }
    }
}
```

Código 9: ejemplo de uso de ciclos para iterar sobre conjuntos.

3.2. Ciclo *while*

El ciclo *while* es fundamentalmente una estructura para hacer repeticiones de un bloque de código mientras que alguna condición se cumpla; es decir, mientras que alguna expresión, al ser evaluada, de verdadero. Un ejemplo común puede ser leer datos mientras haya datos para leer o hasta que se ingrese cierto valor. El ciclo *while* consta de 4 partes, las cuales se ven en el Código 10 resaltadas con comentarios, como se hizo con el ciclo *for*, y luego sin los comentarios, para mayor claridad.

1. /*1*/: la primera parte la componen las instrucciones que están antes del ciclo, y se ejecuta una sola vez. En el ejemplo del Código 10 es un mensaje y una instrucción.
2. /*2*/: la segunda parte está dentro de la estructura y es una expresión que al ser evaluada debe dar un valor de verdad. Cuando este valor es *true*, se ejecuta el bloque que está dentro del ciclo (parte 3) y vuelve a evaluarse. En el Código 10 es (n = sn.nextInt()) >0, que es básicamente una lectura, seguida de una asignación y después una comparación de que lo que se leyó sea mayor a 0.
3. /*3*/: la tercera parte es el bloque del ciclo; en el ejemplo del Código 10 es una impresión a la salida estándar: System.out.println(n);
4. /*4*/: la cuarta parte es, finalmente, lo que está después del ciclo, que en el ejemplo del código mencionado es un mensaje.

La secuencia de instrucciones, si la pudiéramos enumerar, como se hizo para explicar el ciclo *for*, se vería algo como: 1,2,3, 2,3, 2,3, 2,3, 2,3, ... ,2, 4.

3.2.1. Ejemplo 9

En el Código 10 se puede apreciar un programa que lee números enteros, mientras estos sean enteros positivos, y los imprime a la salida estándar. Cuando se le ingresa un número negativo o 0, la condición del *while* es falsa y el flujo de programa sigue con las instrucciones después del *while*.

```
import java.util.Scanner;
public class WhileExample{
    public static void main( String args[]){
        Scanner sn = new Scanner(System.in);
        int n;
        System.out.println("Before the while");
        /*1*/
        while( /*2*/(n = sn.nextInt()) > 0 )
        {
            /*3*/
            System.out.println(n);
        }
        /*4*/
        System.out.println("After the while");

        System.out.println("Before the while");
        while( (n = sn.nextInt()) > 0 )
        {
            System.out.println(n);
        }
        System.out.println("After the while");
        sn.close();
        sn.close();
    }
}
```

Código 10: ejemplo uso del ciclo *while*

3.3. Ciclo *do-while*

El ciclo *do-while* es fundamentalmente una estructura que se usa para hacer repeticiones de un bloque de código hasta que alguna condición deje de cumplirse. O, visto de otra manera, hacer repeticiones (al menos una) mientras cierta condición se cumpla; es decir, mientras que alguna expresión al ser evaluada de verdadero. Un ejemplo común puede ser leer datos hasta que haya datos para leer (sabiendo que habrá al menos un dato), o hasta que se ingrese cierto valor. El ciclo *do-while* consta de 4 partes, al igual que el ciclo *while*, las cuales se ven en el código 11 resaltadas con comentarios, como se hizo con el ciclo *for*, y luego sin los comentarios, para mayor claridad.

1. /*1*/: la primera parte la componen las instrucciones que están antes del ciclo, y se ejecuta una sola vez. En el ejemplo del Código 11 es un mensaje y una instrucción.

2. /*2*/: la segunda parte es el bloque del ciclo; en el ejemplo en el Código 10 es una lectura de dato y una impresión a la salida estándar: n = sn.nextInt(); y System.out.println(n);. Es importante notar que en el caso del *do-while*, a diferencia del *while*, la parte del bloque del ciclo se ejecuta al menos una vez, puesto que la expresión es booleana; la condición del ciclo se ejecuta después de la primera vez de ejecución del bloque.

3. /*3*/: la tercera parte está dentro de la estructura y es una expresión que al ser evaluada debe dar un valor de verdad; cuando este valor es *true*, se ejecuta el bloque que está dentro del ciclo (parte 2) y vuelve a evaluarse. En el Código 11 es n >0, que es básicamente una lectura, seguida de una asignación y después una comparación de que lo que se leyó es mayor a 0.

4. /*4*/: la cuarta parte es lo que está después del ciclo que, en el ejemplo del Código mencionado, es un mensaje.

La secuencia de instrucciones, si la pudiéramos enumerar, como se hizo para explicar el ciclo *for*, se vería algo como: 1,2, 3,2, 3,2, 3,2, 3,2, ... ,3, 4.

3.3.1. Ejemplo 10

```
import java.util.Scanner; public
class DoWhileExample{
    public static void main( String args[]){
        Scanner sn = new Scanner(System.in);
        int n;
        System.out.println("Before the while");
        /*1*/
        do
        {
            /*2*/
            n = sn.nextInt();
            System.out.println(n);
        }while( /*3*/ n > 0 );
        /*4*/
        System.out.println("After the while");


        System.out.println("Before the while");
        do
        {
            n = sn.nextInt();
            System.out.println(n);
        }while( n > 0 );
            System.out.println("After the while");
        sn.close();
    }
}
```

Código 11: ejemplo uso del ciclo *do-while*

3.4. Equivalencia de ciclos

Aunque cada ciclo tiene sus particularidades y usos recomendados, hay que entender y tener en cuenta que lo que se pueda hacer con uno, se puede hacer con los otros con relativa facilidad y emplear en algunos casos un condicional, una o dos líneas de código adicionales, como se ve en el siguiente ejemplo.

3.4.1. Ejemplo 11

En el Código 12 se aprecia cómo con los tres ciclos se puede hacer una misma tarea y, además, cómo cada uno se puede adaptar para hacer tareas del tipo que son recomendadas para alguno de los otros ciclos. En este ejemplo se omiten bastantes llaves para que se pueda apreciar todo en una sola página.

```
public class AllLoopsExample{
    public static void main( String args[ ]){
        java.util.Scanner sn = new java.util.Scanner( System.in );
        System.out.println("For loop common use:");
        for( int i = 0; i < 5 ; ++ i)
            System.out.println(i);
        System.out.println("While loop as a for loop:");
        int i = 0;
        while( i < 5 )
            System.out.println(i++);
        System.out.println("Do-while loop like a for loop:");
        i = 0;
        do{
            if( i < 5 )System.out.println(i);
            ++i;
        }while(i < 5);
        System.out.println("While loop common use:");
        int x;
        while((x = sn.nextInt()) > 0)
            System.out.println(x);
        System.out.println("For loop as a while loop:");
        for(int x2;(x2 = sn.nextInt()) > 0;)
            System.out.println(x2);
        System.out.println("Do-while loop as a while loop:");
        int x4;
        do{
            x4 = sn.nextInt();
            if( x4 > 0 )System.out.println(x4);
        }while(x4 > 0);
        System.out.println("Do-while loop common use:");
        do{
            System.out.println(x4 = sn.nextInt());
        }while( x4 > 0 );
        System.out.println("For loop as a do-while loop:");
        int x5 = sn.nextInt();
        System.out.println(x5);
        for (; x5 > 0 ;)
            System.out.println(x5 = sn.nextInt()); System.out.println("While
        loop as a do-while loop:");
        int x6 = sn.nextInt();
        System.out.println(x6);
        while ( x6 > 0 )
            System.out.println(x6 = sn.nextInt());
        sn.close();
    }
}
```

Código 12: ejemplo de equivalencia entre ciclos

4. Sentencias de interrupción de ciclos

Adicional a las estructuras de repetición (ciclos) y la posibilidad de poner dentro de ellas estructuras de control como el condicional *if*, en algunos lenguajes hay sentencias para interrumpir la ejecución de un ciclo: la sentencia *break* y la sentencia *continue* son dos de ellas, presentes en Java.

4.1. Sentencia *break*

La sentencia *break*, al estar en el flujo de programa dentro de un ciclo, hace que el flujo de programa siga en la primera instrucción después del ciclo en que se encuentra; por esto se entiende como un rompa el ciclo.

4.1.1. Ejemplo 12

En el Código 13 se pueden ver dos ejemplos de uso de *break* para interrumpir la ejecución de ciclos. En el primero, el cual es un solo ciclo *for*, al evaluarse cierta la condición del *if*, cuando *i* es igual a tres, se ejecuta la instrucción *break* y el ciclo se acaba.

En el segundo ejemplo, donde se muestran dos ciclos anidados, la sentencia *break* se ejecuta varias veces en el ciclo *for* interno y una vez en el ciclo *while* externo. Es recomendable hacer un seguimiento paso a paso de este programa para un mejor entendimiento de las posibilidades de uso de la sentencia *break*.

```
public class BreakExample{
    public static void main( String args[]){
        System.out.println("Break example:");
        for( int i = 0; i < 1000; i ++ ){
            if( i == 3 ){
                break;
            }
            System.out.println("In the loop: i = "+i);
        }
        System.out.println("After the for loop.");
        int whileIteration = 0;
        System.out.println("Break example with nested loops:");
        while (true) {
            whileIteration++;
            for( int i = 0; i < 10; i ++ ){
                if( i == 3 ){
                    break;
                }
                System.out.println("In the loop: i = "+i);
            }
            System.out.println("After the for loop, inside the while loop.");
            if( whileIteration == 3 )
                break;
        }
        System.out.println("At the end of the program.");
    }
}
```

Código 13: Ejemplo de uso de *break*.

4.2. Sentencia *continue*

La sentencia *continue*, al ser encontrada en el flujo de programa dentro de un ciclo, hace que el flujo de programa siga en la siguiente iteración; por esto se entiende como un continúe en la siguiente iteración.

4.2.1. Ejemplo 13

En el Código 14 hay dos ejemplos de uso de *continue* para interrumpir la ejecución de ciclos. En el primero, el cual es un solo ciclo *for*, al evaluarse cierta la condición del *if*, cuando *i* es igual a tres, se ejecuta la instrucción *continue*, y el ciclo pasa inmediatamente a la iteración siguiente, si ejecutar lo que queda del bloque del *for* que es una impresión, cuando *i* vale 3.

En el segundo ejemplo, donde se muestran dos ciclos anidados, la sentencia *continue* se ejecuta varias veces en el ciclo *for* interno y una vez en el ciclo *for* externo. Es necesario hacer un seguimiento paso a paso de este programa para un mejor entendimiento de las posibilidades de uso de la sentencia *continue*.

```
public class ContinueExample{
    public static void main( String args[]){
        System.out.println("Continue example:");
        for( int i = 0; i <= 5; i ++ ){
            if( i == 3 ){
            continue;
            }
            System.out.println("In the loop: i = "+i);
        }
        System.out.println("After the for loop.");
        System.out.println("Continue example with nested loops:"); for( int
        k = 0; k <= 5; ++k ){
            if( k == 3 )
                continue;
            for( int i = 0; i < 10; i ++ ){
                if( i == 3 ){
                continue;
                }
                System.out.println("In the loop: i = "+i);
            }
            System.out.println("After the for loop, inside the while loop.");
        }
        System.out.println("At the end of the program.");
    }
}
```

Código 14: ejemplo de uso de *continue*.

5. Usos especiales de variables en ciclos

En esta sección se exponen tres usos muy comunes de variables en programas. Estos usos son tan corrientes, que han sido clasificados de manera amplia por la comunidad de programadores de lenguajes como claramente distinguibles de otros usos que se le den a las variables. Estos usos son: para contar, para acumular y para detectar eventos o condiciones.

5.1. Variables tipo contador

Las variables tipo contador sirven fundamentalmente para contar cosas o iterar en un ciclo (generalmente *for*). Es muy común usar las variables i, j y k en ciclos como el contador del ciclo; se les da este nombre porque van contando las iteraciones del ciclo. Una variable tipo contador es entonces de tipo entero, que sirve para contar de uno en uno, disminuyendo de uno en uno o aumentando en cierta cantidad fija hasta que se cumpla una condición. Nombres comunes y buenos para tener en cuenta en prácticas de programación son *cnt*, *count*, *counter*, *errorCounter*, *errorCnt*, entre otros.

5.1.1. Ejemplo 14

En el Código 15 hay un ejemplo de uso de contadores, en el cual se cuentan las posibles combinaciones de valores de tres variables entre 0 y 9, que, al sumarlas, el resultado da un múltiplo de 7.

```
public class CountExample{
    public static void main( String args[]){
        int n = 10;
        int cnt = 0;
        for( int i = 0; i < n; ++ i ){
            for (int j = 0; j < n; j++){
                for (int k = 0; k < n; k++){
                    if( (i + j + k)%7 == 0 ){
                        ++cnt;
                    }
                }
            }
        }
        System.out.println("cnt = "+cnt);
    }
}
```

Código 15: ejemplo de uso de variables como contadores.

5.1.2. Variables tipo acumulador

El segundo uso común de variables son las usadas como acumuladores. A diferencia de las de uso como contadores, estas se pueden encontrar en tipos enteros, reales e incluso booleanos. Se usan cuando se desean acumular sumas de magnitudes, o productorias o secuencias de conjunciones booleanas o disyunciones booleanas. Nombres comunes y buenos para tener en cuenta en prácticas de programación son *accum*, *accumulator*, *areaAccum*, *moneyAccum*, entre otros.

5.1.3. Ejemplo 15

En el Código 16 hay un ejemplo muy similar al mostrado en el Código 15, pero en lugar de contar los múltiplos que se encuentran, el programa suma todos sus valores para mostrar el resultado al final.

```
public class AccumExample{
    public static void main( String args[]){
        int n = 10;
        int accum = 0;
        for( int i = 0; i < n; ++ i ){
            for (int j = 0; j < n; j++){
                for (int k = 0; k < n; k++){
                    if( (i + j + k)%7 == 0 ){
                        accum += i+j+k;
                    }
                }
            }
        }
        System.out.println("accum = "+accum);
    }
}
```

Código 16: Ejemplo de uso de variables como acumuladores

5.2. Variables tipo bandera

En algunas ocasiones, es de interés detectar si una condición se cumple o si algún valor ocurre en una serie de iteraciones u operaciones. Cuando se necesita recorrer o analizar un conjunto de posibilidades y decir al final si ocurrió o no alguna posibilidad o conjunto pequeño de posibilidades, se usan variables como banderas. Estas variables son generalmente de tipo booleano y se tienden a declarar falsas antes de analizar el conjunto de posibilidades; en el caso de detectar algo, se les cambia su valor a *true*, para proceder a tomar acciones respecto a lo que se encontró. También pueden ser de tipo entero cuando las posibilidades no se limitan a que existe o no una situación, sino que son una de varias posibilidades en un conjunto pequeño de estas. Así, por ejemplo, se podría inicializar una variable bandera en -1 y durante el análisis del conjunto de posibilidades darle valores de 0, 1 o 2, etc., según se necesite, para después tomar acciones para cada detección.

5.2.1. Ejemplo 16

En el Código 17 hay un ejemplo de uso de una variable como bandera para detectar algo. En este caso, se desea detectar si alguna combinación de valores de i, j y k, entre 0 y 9, tiene como suma un número que es a la vez múltiplo de 7, 5 y 3.

```
public class Flags{
    public static void main( String args[]){
        int n = 10;
        boolean flag = false;
        for( int i = 0; i < n && !flag; ++ i ){
            for (int j = 0; j < n && !flag; j++){
                for (int k = 0; k < n && !flag; k++){
                    if( ( i + j + k ) % 7 == 0 &&
                        ( i + j + k ) % 5 == 0
                        &&( i + j + k ) % 3 ==
                        0){ flag = true;
                    }
                }
            }
        }
        System.out.println("Flag = "+flag);
    }
}
```

Código 17: ejemplo de uso de variables como banderas

6. División de cadenas

Para finalizar, se expone el uso de ciclos en conjunción con otra tarea muy común en el diseño de programas y algoritmos: partir cadenas. Los lenguajes de programación generalmente tienen al menos una manera de hacer esta tarea. En el caso de Java hay varias; a continuación se muestran dos:

6.1. Uso de *StringTokenizer*

Una de las herramientas disponibles en Java para dividir cadenas de caracteres es la clase *StringTokenizer*, a la cual se le puede pasar un valor tipo *String* que sirva de separador; en el caso de que nada se pase en su instanciación, la separación se hará por espacios en blanco: espacios, tabulaciones, saltos de línea y retornos de carro. Para más detalle, se debe siempre recordar la revisión de la documentación relacionada (ver Oracle, 1995b).

El Código 18 es un ejemplo de uso de *StringTokenizer* para dividir cadenas. Hay varios aspectos adicionales que mencionar en este ejemplo. El primero es que toda la lectura se hace línea a línea, inclusive la del entero, que se lee al inicio del programa (se podría haber hecho con *nextInt*). Esto porque, en general, para evitar posibles errores sencillos de programación, una buena recomendación es seguir el principio de **si en la lectura se leerán algunas líneas completas, entonces leer líneas completas en toda la lectura.**

Otro aspecto a tener en cuenta es el uso de la clase *StringBuilder* de Java, que permite operar cadenas de manera mucho más eficiente que usar concatenaciones y operaciones con la clase *String*. Para más detalles de uso de la clase *StringBuilder*, revisar la documentación de Java Oracle (1995ª).

Lo que hace el Código 18 es básicamente leer una serie de líneas y, usando *StringTokenizer* y *StringBuilder*, construir una cadena de lo leído sin espacios y con todo lo leído en minúsculas. Se recomienda, como siempre en todos los ejemplos de código de este módulo, hacer un seguimiento paso a paso al programa y entender cada instrucción.

```
import java.util.Scanner;
import java.util.StringTokenizer;
public class StringTokenizerExample{
    public static void main( String args[]){
        Scanner sn = new Scanner( System.in );
        String line;
        StringBuilder sb = new StringBuilder();
        System.out.print("How many lines should the program read: ");
        int n = Integer.parseInt( sn.nextLine());
        System.out.println("Type "+n+" lines: ");
        for( int i = 1; i <= n; ++i ){
            System.out.print("Type line "+i+" : ");
            line = sn.nextLine();
            System.out.println("Line we just read: _"+line+"_");
            StringTokenizer st = new StringTokenizer(line);
            while( st.hasMoreTokens() ){
                sb.append( st.nextToken().toLowerCase() );
            }
        }
        System.out.println("All the input in lower case, and without spaces or tabs.");
        System.out.println(sb.toString());
        sn.close();
    }
}
```

Código 18: ejemplo de uso *StringTokenizer*.

6.2. Uso de *Split*

Otra forma de separar cadenas es el uso del método ***split*** de la clase *String*. Este método se usa de manera similar a como se instancia la clase *StringTokenizer*. En el ejemplo del Código 19, se muestra un programa muy similar al del ejemplo de *StringTokenizer*, pero en lugar de usar este, se emplea el método ***split*** para dividirlas. En este ejemplo de utilización de *split* se puede ver un uso de *for* que no se ha expuesto hasta el momento. Este *for* es conocido como *foreach* y sirve para iterar con una variable temporal a través de los elementos de un arreglo o colección. En la siguiente unidad este tema se tratará con más detalle.

```
import java.util.Scanner;
import java.util.StringTokenizer; public
class StringSplitExample{
    public static void main( String args[]){
        Scanner sn = new Scanner( System.in );
        String line;
        StringBuilder sb = new StringBuilder();
        System.out.print("How many lines should the program read: ");
        int n = Integer.parseInt( sn.nextLine());
        System.out.println("Type "+n+" lines: ");
        for( int i = 1; i <= n; ++i ){
            System.out.print("Type line "+i+" : ");
            line = sn.nextLine();
            System.out.println("Line we just read: _"+line+"_");

            for( String s : line.split("[\t\r\n ]+") ){
                sb.append( s.toLowerCase() );
            }
        }
        System.out.println("All the input in lower case, and without spaces or tabs.");
        System.out.println(sb.toString());
        sn.close();
    }
}
```

Código 19: Ejemplo de uso de split para separar cadenas.

7. Problemas propuestos

Todos los problemas de esta sección constan de un enunciado en el que se solicita el diseño e implementación de un programa en Java que tome como entrada ciertos datos leídos desde su entrada estándar y produzca como única salida los datos solicitados, que pueden ser cálculos o transformaciones hechas con esos datos de entrada. En todos los problemas propuestos se debe asumir que las entradas cumplen con lo estipulado en la sección "**`Descripción de entrada`**" de cada problema. Dicho esto, se debe entonces diseñar e implementar un programa que cumpla con lo estipulado en la sección "**`Descripción de salida`**", y que sea únicamente lo solicitado allí, nada más.

1. Contador de dígitos. Hacer un programa que cuente la cantidad de dígitos de un número.
   - Descripción de entrada: un número entero entre -3000000000 y 3000000000.
   - Descripción de salida: un entero positivo indicando la cantidad de dígitos del número leído.
   - Ejemplo de entrada:
     0
   - Ejemplo de salida:
     1
   - Ejemplo de entrada 2:
     123456

- Ejemplo de salida 2:
  6
- Ejemplo de entrada 3:
  -98746
- Ejemplo de salida 3:
  5
- Ejemplo de entrada 4:
  2999999999
- Ejemplo de salida 4:
  10

2. Búsqueda del mayor. Hacer un programa que tome como entrada un número entero positivo N seguido de N números y como salida muestre el mayor de los números.
   - Descripción de entrada: un número positivo entero N seguido de N números enteros.
   - Descripción de salida: el mayor de los N números leídos. Asumir que N no va a ser mayor a 100 y los N números todos estarán entre -3000000000 y 3000000000.
   - Ejemplo de entrada:
     5 200 100 3000 656 98
   - Ejemplo de salida:
     3000
   - Ejemplo de entrada:
     5 -9 -8 -6333333 -9 -2
   - Ejemplo de salida:
     -2
   - Ejemplo de entrada:
     1 -3000000000
   - Ejemplo de salida:
     -3000000000

3. Búsqueda del menor. Hacer un programa que tome como entrada un número entero positivo N seguido de N números y como salida muestre el menor de los números.
   - Descripción de entrada: un número positivo entero N seguido de N números enteros.
   - Descripción de salida: el menor de los N números le´ıdos. Asumir que N no va a ser mayor a 100 y los N números estarán entre -3000000000 y 3000000000.
   - Ejemplo de entrada:
     5 200 100 3000 656 98
   - Ejemplo de salida:
     98
   - Ejemplo de entrada 2:
     5 -9 -8 -6333333 -9 -2
   - Ejemplo de salida 2:
     -6333333
   - Ejemplo de entrada 3:
     1 3000000000
   - Ejemplo de salida 3:
     3000000000

4. Análisis de coordenadas. Diseñar un programa que lea una secuencia de parejas de números (no se sabe cuántas) hasta que le ingresen la pareja 0 0. El programa debe asumir que son coordenadas en el plano cartesiano (coordenadas enteras, solo leerá enteros). Después de la lectura, el programa debe imprimir el par ordenado más cercano al origen, el más lejano al origen y el promedio de las distancias al origen de todos los puntos. Además, el programa debe mostrar un mensaje indicando si se ingresaron puntos sobre los ejes o no (coordenada x=0 o y=0). Se puede asumir que en la entrada de datos no hay parejas ordenadas con la misma distancia al origen. Se deben ignorar los puntos que están en el tercer cuadrante (x¡0 y y¡0).

- Descripción de entrada: parejas de números enteros hasta que se lea la pareja 0 0.
- Descripción de salida: XXXXX.
- Ejemplo de entrada:
  5
  63 65
  1 1
  5 5
  2 3
  20 20
- Ejemplo de salida:
  Closest to origin: 1 1
  Furthest to origin: 63 65
  Average distance to origin: 26.179163951646263

5. Contar dígitos en intervalo. Dados dos números A y B, diseñar e implementar un programa que cuente la cantidad de veces que se escribiría cada dígito si se escribieran todos los números entre A y B.

- Description de entrada: two integers A and B. Both between -1000 y 1000.
- Descripción de Salida: Un mensaje por cada d´ıgito entre 0 y 9 indicando cuántas veces se imprimir´ıa.
- Ejemplo de entrada:
  -1 1
- Ejemplo de salida:
  Digit 0 is printed 1 times.
  Digit 1 is printed 2 times.
  Digit 2 is printed 0 times.
  Digit 3 is printed 0 times.
  Digit 4 is printed 0 times.
  Digit 5 is printed 0 times.
  Digit 6 is printed 0 times.
  Digit 7 is printed 0 times.
  Digit 8 is printed 0 times.
  Digit 9 is printed 0 times.
- Ejemplo de entrada 2:
  654 123
- Ejemplo de salida 2:
  Digit 0 is printed 103 times.
  Digit 1 is printed 180 times.
  Digit 2 is printed 210 times.
  Digit 3 is printed 214 times.
  Digit 4 is printed 214 times.
  Digit 5 is printed 208 times.

Digit 6 is printed 158 times.
Digit 7 is printed 103 times.
Digit 8 is printed 103 times.
Digit 9 is printed 03 times.

6. Seno de ángulo. Diseñe un método que calcule el seno de un ángulo usando series de *Taylor*. Nota: no usar métodos de la clase *Math* de Java. Para este problema vale la pena calcular términos mientras sean significativos según alguna tolerancia preestablecida; por ejemplo, una tolerancia de 0.0000000001.
   - Descripción de entrada: un valor real, representando un ángulo en radianes para calcular el seno.
   - Descripción de salida: el seno del ángulo leído en radianes.
   - Ejemplo de entrada:
     2
   - Ejemplo de salida:
     0.9092974268256817
   - Ejemplo de entrada 2:
     1.57
   - Ejemplo de salida 2:
     0.9999996829318346

7. Coseno de ángulo. Diseñe un método que calcule el coseno de un ángulo usando series de *Taylor*. Nota: no usar métodos de la clase *Math* de Java. Para este problema vale la pena calcular términos mientras sean significativos según alguna tolerancia preestablecida; por ejemplo, una tolerancia de 0.0000000001.
   - Descripción de entrada: un valor real, representando un ángulo en radianes para calcular el seno.
   - Descripción de salida: el seno del ángulo leído en radianes.
   - Ejemplo de entrada:
   - 2
   - Ejemplo de salida
   - 0.4161468365471424
   - Ejemplo de entrada 2:
   - 1.57
   - Ejemplo de salida 2:
   - 0.0007963267107332633

8. Número primo. Diseñar un programa que tome como entrada un número entero positivo y diga si es primo o no.
   - Descripción de entrada: un número entero positivo mayor o igual a 2 y menor o igual a 2000000.
   - Descripción de salida: *prime* o *not prime*, según el caso.
   - Ejemplo de entrada:
     7
   - Ejemplo de salida:
     prime
   - Ejemplo de entrada 2:
     2113
   - Ejemplo de salida 2:
     prime
   - Ejemplo de entrada:
     70

- Ejemplo de salida:
  not prime
- Ejemplo de entrada 2:
  21135
- Ejemplo de salida 2:
  not prime

9. Números primos en intervalo. Diseñar e implementar un programa que tome como entrada dos números enteros positivos mayores o iguales que 2 y menores o iguales que 20000, y muestre a la salida estándar cuántos números primos hay en ese intervalo.

- Descripción de entrada: una pareja de números enteros positivos.
- Descripción de salida: un número entero no negativo indicando la cantidad de números primos en el intervalo dado.
- Ejemplo de entrada:
  8 10
- Ejemplo de salida:
  0
- Ejemplo de entrada 2:
  10 20
- Ejemplo de salida 2:
  4
- Ejemplo de entrada 3:
  11 19
- Ejemplo de salida 3:
  4

10. Subconjuntos de tamaño fijo. Modificar el ejemplo del Código 9 para que, en lugar de tener fija la cadena, la lea usando la entrada estándar. En el caso de que la cadena leída sea de mayor tamaño que 15 o que tenga elementos repetidos, debe mostrar un mensaje de error. En otro caso, debe leer un número entero **k** y seguido a esto mostrar todos los subconjuntos del conjunto representado que tengan tamaño **k**.
11. Descripción de entrada: una cadena y posiblemente un número entero **k**, dependiendo de si la cadena
12. tiene solo elementos diferentes.
13. Descripción de salida: todos los subconjuntos de tamaño **k** que tiene el conjunto representado por la cadena leída.
14. Ejemplo de entrada:
xyzwab 3
15. Ejemplo de salida:

{x,y,z}
{x,y,w}
{x,z,w}
{y,z,w}
{x,y,a}
{x,z,a}
{y,z,a}
{x,w,a}
{y,w,a}
{z,w,a}
{x,y,b}
{x,z,b}
{y,z,b}
{x,w,b}
{y,w,b}
{z,w,b}
{x,a,b}
{y,a,b}
{z,a,b}
{w,a,b}

11. Hacer un seguimiento paso a paso al siguiente programa, asumiendo que se lee el código estudiantil o número de cédula. Recomendación: usar una hoja de cálculo o papel y lápiz. Después de hacer el seguimiento paso a paso, correr el programa con la entrada que se hizo el seguimiento y verificar que todas las salidas correspondan a las detectadas durante este.

```
import java.util.Scanner;
public class MainLoops{
    public static void main (String[] args) {
        Scanner sn = new Scanner(System.in); int
        cc = sn.nextInt() % 10;
        int from = cc-6;
        int to = cc;
        boolean parity = true;
        boolean positive = true;
        int iterCounter = 0;
        int accum = 0;
        for( int i = from; i <= to; ++i ){
            if( i % 2 == 0 )
                { continue;
            }
            int cnt = 0;
            for( int j = from; j <= to; ++j ){
                if( cnt == 2 ){
                    break;
                }
                ++iterCounter;
                System.out.println(iterCounter+" -> "+i+","+j);
                accum = accum + (i+j);
                positive = positive && ((i+j) > 0);
                parity = parity && ((i+j) % 2==0);
                ++cnt;
            }
        }
        System.out.println("iterCounter ="+iterCounter);
        System.out.println("accum ="+accum);
        System.out.println("positive ="+positive);
        System.out.println("parity ="+parity); sn.close();
    }
}
```

Código 20: seguimiento a programa paso a paso.

8. Extras

8.1. Ejemplos de indentaciones correctas para el ejemplo 8

Para propósitos de este módulo, se presentan ejemplos de lo que se considera una buena indentación de código. Vale la pena analizar con el tutor las ventajas y desventajas de cada manera de editar. En particular, se puede notar las maneras de indentar en los ejemplos 21, 22, 23 y 24, que se basan en el ejemplo 9, son variaciones de los estilos dados por *stroustrup* y *kernigham*. De acuerdo con el equipo de tutores y autores de la Institución

Universitaria Politécnico Grancolombiano, son una muy buena aproximación a cómo debe ser el indentado de códigos fuente en este y otros módulos relacionados.

```
import java.util.Arrays;
public class StringBitsAndSetsExample
{
    public static void main(String[] args){
        String s = "abcdef";
        int n = s.length();
        int limit = 1<<n;
        String arr[] = s.split("");
        System.out.println("Print all subsets of: "+Arrays.toString(arr));
        for( int mask = 0; mask < limit; ++ mask )
        {
            System.out.print("{ ");
            for (int bit = 0; bit < n; bit++){
                boolean isIn = (mask & (1<<bit) ) > 0;
                if( isIn )
                {
                    System.out.print(s.charAt(bit)+" "); }
            }
            System.out.println("}");
        }
    }
}
```

Código 21: ejemplo de correcta indentación 1

```
import java.util.Arrays;
public class StringBitsAndSetsExample2{
    public static void main(String[] args)    {
        String s = "abcdef";
        int n = s.length();
        int limit = 1<<n;
        String arr[] = s.split("");
        System.out.println("Print all subsets of: "+Arrays.toString(arr));
        for( int mask = 0; mask < limit; ++ mask ){
            System.out.print("{ ");
            for (int bit = 0; bit < n; bit++){
                boolean isIn = (mask & (1<<bit) ) > 0;
                if( isIn ){
                    System.out.print(s.charAt(bit)+" "); }
            }
            System.out.println("}");
        }
    }
}
```

Código 22: ejemplo de correcta indentación 2

```
import java.util.Arrays;
public class StringBitsAndSetsExample3{
    public static void main(String[] args)    {
        String s = "abcdef";
        int n = s.length();
        int limit = 1<<n;
        String arr[] = s.split("");
        System.out.println("Print all subsets of: "+Arrays.toString(arr));
        for( int mask = 0; mask < limit; ++ mask ){
            System.out.print("{ ");
            for (int bit = 0; bit < n; bit++)
                if( (mask & (1<<bit) ) > 0 )
                    System.out.print(s.charAt(bit)+" ");
            System.out.println("}");
        }
    }
}
```

Código 23: Ejemplo de correcta indentación 3

```
import java.util.Arrays;
public class StringBitsAndSetsExample4 {
    public static void main(String[] args){
        String s = "abcdef";
        int n = s.length();
        int limit = 1<<n;
        String arr[] = s.split("");
        System.out.println("Print all subsets of: "+Arrays.toString(arr));
        for( int mask = 0; mask < limit; ++ mask )
        {
            System.out.print("{ ");
            for (int bit = 0; bit < n; bit++)
                if( (mask & (1<<bit) ) > 0 )
                    System.out.print(s.charAt(bit)+" ");
            System.out.println("}");
        }
    }
}
```

Código 24: ejemplo de correcta indentación 4

Referencias

Google. (2018). *Google Java Style Guide*. Recuperado de https://google.github.io/styleguide/javaguide.html

Oracle. (1995a). *Class StringBuilder*. Recuperado de https://docs.oracle. com/javase/7/docs/api/java/lang StringBuilder.html

Oracle. (1995b). *ClassStringTokenizer.* Recuperado de https://docs.oracle. com/javase/7/docs/api/java/util/ StringTokenizer.html

Oracle. (1999a). *Code Conventions for the Java TM Programming Language.* Recuperado de http:// www.oracle.com/technetwork/java/codeconvtoc-136057.html

Oracle. (1999b). *How to Write Doc Comments for the Javadoc Tool*. Recuperado de http://www.oracle.com/ technetwork/articles/java/index-137868.html

INFORMACIÓN TÉCNICA

Ingeniería y Ciencias Básicas

Módulo: Conceptos Fundamentales de Programación
Unidad 1: Introducción a la programación
Escenario 2: Estructuras de control de flujo de programa

Autor: Rafael Niquefa

Asesor Pedagógico: Angie Laitón
Diseñador Gráfico: Rafael Niquefa
Asistente: Alejandra Morales

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.